

AI 个性化学习路径系统 — Mac 部署指南

版本: v1.0 · 2026-05-27

项目: ai-learning-path (北科大 MBA 人工智能课 L-2 作业)

仓库: <https://github.com/haloZh/ai-learning-path>

1. 运行环境要求

组件	最低版本	推荐版本	说明
macOS	12 Monterey	14 Sonoma	均可运行
Python	3.11	3.12	系统自带版本可能过旧, 建议重装
Node.js	20 LTS	22 LTS	构建前端
Git	2.30+	最新版	macOS 自带, 或通过 Homebrew 更新
磁盘空间	5 GB	8 GB	bge-m3 模型约 2.3 GB
内存	8 GB	16 GB	首次加载 RAG 模型

注意: Mac 系统自带的 `python3` 版本可能是 3.9 或更旧, 建议通过 Homebrew 安装 3.12。

2. 第一步: 安装基础软件

2.1 安装 Homebrew (包管理器)

打开终端 (Terminal), 执行:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

安装完成后, 按照终端提示将 brew 加入 PATH (Apple Silicon Mac 需要额外执行):

```
# Apple Silicon (M1/M2/M3) 执行这一步
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
eval "$(/opt/homebrew/bin/brew shellenv)"
```

验证:

```
brew --version
```

2.2 安装 Python 3.12

```
brew install python@3.12
```

```
# 验证
```

```
python3.12 --version
```

```
# 应输出: Python 3.12.x
```

安装后系统可能同时存在多个 Python 版本, 后续步骤统一使用 `python3.12` 命令创建虚拟环境, 进入虚拟环境后直接用 `python` 即可。

2.3 安装 Node.js 22 LTS

```
brew install node@22
```

```
# 验证
```

```
node --version
```

```
# 应输出: v22.x.x
```

```
npm --version
```

2.4 确认 Git

macOS 自带 Git, 通常足够使用:

```
git --version
```

```
# 如果提示安装 Xcode Command Line Tools, 按提示安装即可
```

如需升级:

```
brew install git
```

3. 第二步：配置 SSH Key

3.1 生成密钥

```
ssh-keygen -t ed25519 -C "your_email@example.com"
# 一路回车（默认路径 ~/.ssh/id_ed25519，不设密码）
```

3.2 复制公钥

```
cat ~/.ssh/id_ed25519.pub
# 复制输出的整行内容（ssh-ed25519 AAAA... your_email）
```

或使用命令直接复制到剪贴板：

```
pbcopy < ~/.ssh/id_ed25519.pub
```

3.3 添加到 GitHub

1. 打开 <https://github.com/settings/ssh/new>
2. Title：随便填（如 "My MacBook"）
3. Key type： **Authentication Key**
4. Key： 粘贴刚才复制的内容
5. 点 **Add SSH key**

3.4 验证

```
ssh -T git@github.com
# 输入 yes 后出现：Hi haloZh! You've successfully authenticated...
```

4. 第三步：获取代码

```
# 切换到工作目录（例如桌面）
cd ~/Desktop

# 克隆仓库
git clone git@github.com:haloZh/ai-learning-path.git

# 进入项目目录
cd ai-learning-path
```

5. 第四步：配置 Python 虚拟环境

```
# 使用 Python 3.12 创建虚拟环境
python3.12 -m venv .venv

# 激活虚拟环境（注意：Mac/Linux 用 source，路径用斜杠）
source .venv/bin/activate

# 激活成功后，命令提示符前缀显示（.venv）
```

与 Windows 的区别： - Windows: `.venv\Scripts\Activate.ps1` - Mac/Linux: `source .venv/bin/activate`

安装 Python 依赖

```
pip install -r requirements.txt
```

安装过程约 3-5 分钟。如果遇到编译错误，先安装 Xcode Command Line Tools：

```
xcode-select --install
```

6. 第五步：配置环境变量

```
# 复制配置模板
cp .env.example .env

# 用文本编辑器打开
open -e .env

# 或使用 VSCode: code .env
```

`.env` 文件关键字段说明：

字段	说明	示例值
<code>ARK_API_KEY</code>	火山方舟 API 密钥（必填）	<code>your-key-here</code>
<code>ARK_BASE_URL</code>	方舟 API 地址（已预填）	<code>https://ark.cn-beijing.volces.com/api/v3</code>
<code>ARK_MODEL</code>	模型 ID（已预填）	<code>ep-xxxxx-xxxxx</code>

获取 API Key：登录 <https://console.volcengine.com/ark>，创建推理接入点，复制 Endpoint ID 填入 `ARK_MODEL`，API Key 填入 `ARK_API_KEY`。

7. 第六步：初始化数据库

```
# 1. 导入知识点概念
python -m scripts.seed_concepts

# 2. 导入题目数据（替换为实际xlsx 路径）
python -m scripts.import_questions ~/Desktop/math_questions.xlsx

# 3. 生成学习资源（调用 LLM，约 1-2 分钟）
python -m scripts.seed_resources
```

8. 第七步：初始化 RAG 向量库

首次运行需下载 bge-m3 模型（2.3 GB），请确保网络畅通。

```
# 设置国内镜像加速（本次终端会话有效）
export HF_ENDPOINT=https://hf-mirror.com

# 初始化 RAG（首次约 10-20 分钟，之后秒级）
python -m scripts.init_rag
```

模型缓存在 `~/.cache/huggingface/`，后续无需重复下载。

如需每次自动设置镜像，可加入 shell 配置文件：

```
echo 'export HF_ENDPOINT=https://hf-mirror.com' >> ~/.zprofile
```

9. 第八步：构建前端

```
cd frontend
npm install
npm run build
cd ..
```

10. 第九步：启动服务

```
python -m uvicorn app.main:app --port 8000
```

启动成功后打开浏览器访问：<http://localhost:8000>

11. 常见问题

Q1: `python` 命令找不到, 提示 `command not found`

Mac 默认只有 `python3`, 没有 `python`。进入虚拟环境后会自动创建 `python` 别名, 确保先执行:

```
source .venv/bin/activate
```

Q2: `brew` 安装后仍然找不到命令 (Apple Silicon)

```
eval "$(/opt/homebrew/bin/brew shellenv)"  
# 并将上面这行加入 ~/.zprofile 永久生效
```

Q3: `pip install` 时某些包编译失败

```
xcode-select --install  
# 安装完成后重新执行 pip install -r requirements.txt
```

Q4: `init_rag` 下载速度慢

```
# 确认镜像已设置  
echo $HF_ENDPOINT  
# 应输出: https://hf-mirror.com
```

Q5: LLM 调用超时, 系统降级到 `mock` 模式

演示前提前 5 分钟预热: 先访问一次 `/diagnose` 接口, 让 LLM 冷启动完成。如 API Key 正确仍超时, 稍等片刻重试 (火山方舟偶发限速)。

Q6: 端口 8000 被占用

```
# 查找占用进程
lsof -i :8000

# 换一个端口启动
python -m uvicorn app.main:app --port 8001
```

12. 快速启动检查清单

步骤	命令 / 操作	完成
安装 Homebrew	<code>brew --version</code> 验证	☐
安装 Python 3.12	<code>python3.12 --version</code> 验证	☐
安装 Node.js 22	<code>node --version</code> 验证	☐
配置 SSH Key 并添加到 GitHub	<code>ssh -T git@github.com</code> 验证	☐
克隆仓库	<code>git clone ...</code>	☐
创建并激活虚拟环境	<code>source .venv/bin/activate</code>	☐
安装 Python 依赖	<code>pip install -r requirements.txt</code>	☐
配置 <code>.env</code> 填入 API Key	<code>open -e .env</code>	☐
初始化数据库	<code>seed_concepts</code> + <code>import_questions</code> + <code>seed_resources</code>	☐
初始化 RAG 向量库	<code>export HF_ENDPOINT=... && python -m scripts.init_rag</code>	☐
构建前端	<code>npm install && npm run build</code>	☐
启动服务	<code>uvicorn app.main:app --port 8000</code>	☐
访问页面	<code>http://localhost:8000</code>	☐

如遇问题，可参考项目 README 或联系开发团队。